

How to Create Parallel Arrays, Loops And Screen Navigation

File 1 – activity_main.xml (Screen 1

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<!-- LinearLayout stacks everything top to bottom -->
```

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"    // fills full screen width
```

```
    android:layout_height="match_parent"  // fills full screen height
```

```
    android:orientation="vertical"        // stack children top to bottom
```

```
    android:padding="16dp">            // adds space around the edges
```

```
<!-- Title at the top of the screen -->
```

```
<TextView
```

```
    android:id="@+id/tvTitle"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Student Results"
```

```
    android:textSize="22sp"            // sp = scalable pixels, good for text
```

```
    android:textStyle="bold"
```

```
    android:layout_marginBottom="16dp" // space below the title
```

```
<!-- TextView to display the loop output (all students listed) -->
```

```
<TextView
    android:id="@+id/tvStudentList" // ID used in Kotlin to fill this with text
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="16sp"
    android:layout_marginBottom="24dp"
/>
```

<!-- Button to go to Screen 2 -->

```
<Button
    android:id="@+id/btnViewTop" // ID used in Kotlin for click listener
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="View Top Student"
/>
```

File 2 – activity_second.xml (Screen 2)

```
</LinearLayout>
```

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<LinearLayout
```

```
    xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    android:padding="16dp"
```

```
    android:gravity="center">           // centers all children on this screen
```

```
<!-- Heading for Screen 2 -->
```

```
<TextView
```

```
    android:id="@+id/tvHeading"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="wrap_content"
```

```
    android:text="Top Student"
```

```
    android:textSize="22sp"
```

```
    android:textStyle="bold"
```

```
    android:layout_marginBottom="16dp"
```

```
<!-- This TextView will show the student name sent from Screen 1 -->
```

```
<TextView
```

```
        android:id="@+id/tvName"          // Kotlin uses this ID to display the name
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="18sp"
        android:layout_marginBottom="8dp"
    />
```

<!-- This TextView will show the mark sent from Screen 1 -->

```
<TextView
    android:id="@+id/tvMark"          // Kotlin uses this ID to display the mark
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textSize="18sp"
    android:layout_marginBottom="24dp"
/>
```

<!-- Button to go back to Screen 1 -->

```
<Button
    android:id="@+id/btnBack"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Go Back"
/>
```

```
</LinearLayout>
```

File 3 – MainActivity.kt (Screen 1)

// ——— IMPORTS

import android.content.Intent // Intent is used to navigate between screens
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

// ——— CLASS

class MainActivity : AppCompatActivity() {

 // ——— onCreate()

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main) // load activity_main.xml

 // — PARALLEL ARRAYS

 // parallel arrays = two arrays where index 0 of names matches index 0 of marks

 // names[0] = "Mpho" → marks[0] = 85 (they belong together)

 // names[1] = "Aisha" → marks[1] = 92 (same index = same student)

```

val names = arrayOf("Mpho", "Aisha", "Lebo", "Sipho", "Tumi")
    // arrayOf() creates an array — like a list with numbered slots starting at 0

val marks = arrayOf(85, 92, 78, 95, 88)
    // each mark lines up with the name at the same position (same index)

    // — LOOP — build the student list using a for loop —————
    // we use a variable "result" to build up a string of all students
var result = "" // start with empty string, we add to it each loop

    // for loop: i starts at 0, goes up to names.size (5), stops before 5
    // so i = 0, 1, 2, 3, 4 — one loop per student
for (i in 0 until names.size) {

    // names[i] gets the name at position i
    // marks[i] gets the mark at the SAME position i
    // \n adds a new line so each student appears on their own line
    result += "${names[i]}: ${marks[i]}%\n"
}

    // connect to the TextView in XML and set its text to our built string
val tvStudentList = findViewById<TextView>(R.id.tvStudentList)
tvStudentList.text = result // displays the full list on screen

    // — FIND TOP STUDENT using a loop
    _____

    // we loop through the marks array to find the highest mark
var topIndex = 0 // assume index 0 is the top student to start

```

```
for (i in 1 until marks.size) { // start at 1 because 0 is already our starting guess
    if (marks[i] > marks[topIndex]) { // if this mark is higher than current top
        topIndex = i                // update topIndex to this position
    }
}

// after the loop, topIndex holds the position of the highest mark
// topIndex = 3 because marks[3] = 95 (Sipho) is the highest
```

```
// — SCREEN NAVIGATION — go to SecondActivity
```

```
val btnViewTop = findViewById<Button>(R.id.btnViewTop)
```

```
btnViewTop.setOnClickListener {
```

```
    // Intent is like a message that opens a new screen
```

```
    // this = current screen, SecondActivity::class.java = screen to open
```

```
    val intent = Intent(this, SecondActivity::class.java)
```

```
    // putExtra() attaches data to the Intent so Screen 2 can receive it
```

```
    // "STUDENT_NAME" is a KEY — like a label on a package
```

```
    // names[topIndex] is the VALUE — the actual data being sent
```

```
    intent.putExtra("STUDENT_NAME", names[topIndex])
```

```
    intent.putExtra("STUDENT_MARK", marks[topIndex])
```

```
    // you can putExtra() as many values as you need
```

```
    startActivity(intent) // this line actually opens SecondActivity
```

```
}
```


}

}

File 4 – SecondActivity.kt (Screen 2)

// ——— IMPORTS

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

// ——— CLASS

// this is a brand new class – one class per screen, always
class SecondActivity : AppCompatActivity() {

 // ——— onCreate()

 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_second) // loads activity_second.xml

 // — RECEIVE DATA from MainActivity

 // intent holds all the data sent from the previous screen
 // getStringExtra("KEY") fetches a String using the same KEY from putExtra()
 val studentName = intent.getStringExtra("STUDENT_NAME")
 // ?: "Unknown" means: if null (nothing was sent), use "Unknown" as default

```
        // getIntentExtra("KEY", defaultValue) fetches an Int
        // 0 is the default value if nothing was sent
val studentMark = intent.getIntExtra("STUDENT_MARK", 0)
```

```
// — DISPLAY THE DATA on screen
```

```
val tvName = findViewById<TextView>(R.id.tvName)
val tvMark = findViewById<TextView>(R.id.tvMark)

// set the text of each TextView using string interpolation ($variable)
tvName.text = "Name: $studentName" // shows e.g. "Name: Sipho"
tvMark.text = "Mark: $studentMark%" // shows e.g. "Mark: 95%"
```

```
// — GO BACK BUTTON
```

```
val btnBack = findViewById<Button>(R.id.btnBack)

btnBack.setOnClickListener {
    finish() // finish() closes this screen and goes back to the previous one
            // no need for an Intent here — finish() just pops this screen off
}
}
}
```

```
<!-- MainActivity is already here by default, DO NOT remove it -->
```

```
<activity
    android:name=".MainActivity"
```

File 5 — AndroidManifest.xml (register SecondActivity)

```
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

<!-- ADD THIS BLOCK — registers SecondActivity so Android knows it exists -->
<!-- if you skip this, the app crashes when you try to open SecondActivity -->
<activity
    android:name=".SecondActivity"
    android:exported="false" />
<!-- exported="false" means it can only be opened from inside this app -->
```

Here's the order to set it up in Android Studio:

1. Paste activity_main.xml into your existing layout file
2. Create a new layout file called activity_second.xml and paste that in
3. Paste MainActivity.kt into your existing file
4. Create a new Kotlin file called SecondActivity.kt — right click your package folder → New → Kotlin Class → name it SecondActivity
5. Open AndroidManifest.xml and add the SecondActivity block inside the <application> tag

The three key concepts in this app:

- Parallel arrays — names[i] and marks[i] always belong to the same student because they share the same index i
- Loop — for (i in 0 until names.size) goes through every student one by one
- Navigation — Intent carries data from Screen 1 to Screen 2, finish() comes back